

Bilgi Teorisine Dayalı Türkçe Wordle Çözücü

Thomas ACAR

Ağustos 2026

Abstract

Bu doküman, **Wordle TR** kelime çözücüsünün arkasındaki matematiksel ilkeleri ve algoritmik tasarımı açıklamaktadır. 3Blue1Brown'un Bilgi Teorisi ile Wordle çözümü üzerine yayınladığı videodan esinlenerek, Türkçe Wordle oyunu için her aşamada en enformatif tahmini sunan bu çözücüü geliştirdim. Bu açıklama metni, her seviyeden okuyucunun rahatça anlayabileceği şekilde tasarlanmıştır: Wordle'in bir bilgi oyunu olarak nasıl çalıştığını açıklamakta, Bilgi Teorisi ve Shannon Entropisini adım adım örneklerle tanımlamakta ve tüm geçerli kelimeleri aramanın neden yalnızca kalan aday kelimeleri tahmin etmekten daha yüksek bilgi kazancı sağladığını göstermektedir.

1 Giriş

Wordle görünürde 5 harfli basit bir kelime bulmaca oyunudur; ancak arka planda klasik bir **Bilgi Teorisi** problemidir. 3Blue1Brown'un entropi kavramını matematiksel olarak Wordle çözmek için nasıl kullandığını anlatan videosunu izledikten sonra, aynı yaklaşımı Türkçe Wordle için uygulamak istedim.

Bu projenin temel amacı, her geçerli kelimeyi değerlendiren, o kelimenin getirmesi beklenen bilgi miktarını hesaplayan ve gizli hedef kelimeleri en hızlı şekilde eleyen hamleyi öneren hızlı, reaktif bir web uygulaması inşa etmektir.

2 Bir Bilgi Oyunu Olarak Wordle'ı Anlamak

2.1 Kurallar ve Geri Bildirim Sinyali

Wordle oyununda sistem iki ayrı kelime listesi kullanır:

- **Muhtemel Cevaplar Sözlüğü ($\mathcal{A}$):** Olası gizli hedef kelimelerden oluşan seçilmiş küme ($|\mathcal{A}| = 2936$ özgün kelime).
- **Geçerli Tahmin Sözlüğü ($\mathcal{V}$):** Tahmin olarak kabul edilen tüm geçerli kelimelerin bulunduğu geniş sözlük ($|\mathcal{V}| = 5500$ özgün kelime).

Her turda 5 harfli bir $g \in \mathcal{V}$ kelimesi tahmin edilir. Oyun buna karşılık 5 renkli kutucuktan oluşan bir geri bildirim örüntüsü (p) döndürür:

- **Yeşil (2):** Harf doğru ve tam olarak doğru konumda.
- **Sarı (1):** Harf gizli kelimedede var, ancak farklı bir konumda.
- **Gri (0):** Harf gizli kelimedede hiç yok (veya diğer konumda eşleşmiş).

5 kutucuğun her biri için 3 farklı renk seçeneği bulunduğundan, herhangi bir tahmin için en fazla $3^5 = 243$ olası renk kombinasyonu (ya da örüntüsü) mevcuttur.

2.2 Aday Kelime Filtreleme ($\mathcal{C}_k$)

Oyunun başlangıcında (1. Tur), gizli kelime $\mathcal{A}$ kümesindeki 2.936 hedef kelimedenden herhangi biri olabilir. Bu olası kelimeler kümesine **Aday Küme** ($\mathcal{C}_1 = \mathcal{A}$) diyoruz.

İlk tahmininizi ($g_1 \in \mathcal{V}$) yapıp p_1 renk örüntüsünü aldığımızda, $\mathcal{C}_1$ içindeki bu örüntüyü üretmeyecek olan tüm kelimeleri elersiniz. Kalan geçerli hedef kelimeler daha küçük bir $\mathcal{C}_2$ aday kümesini oluşturur.

k . turdaki amacınız, aday kümesini ($\mathcal{C}_k$) mümkün olan en hızlı şekilde tam olarak **1 kelimeye** indirmektir.

—

3 İşin Arkasındaki Matematik: Bilgi Teorisi

3.1 Bilgi Nedir?

Bir hayvanı tahmin etmek için “20 Soru” oyunu oynadığınızı hayal edin:

- “*Tuttuğun hayvan bir memeli mi?*” diye sormak kalan olasılıkları yaklaşık %50/%50 oranında böler. “Evet” veya “Hayır” yanıtı seçeneklerinizi yarı yarıya azaltır. Bu size büyük miktarda bilgi sağlar.
- 1. soruda “*Tuttuğun hayvan bir flamingo mu?*” diye sormak risklidir. Çok yüksek ihtimalle yanıt “Hayır” olacaktır ve bu yanıt neredeyse hiçbir seçeneği elemez.

Bilgi Teorisinde **bilgi, belirsizliğin azalma miktarıdır**. İyi bir hamle, oyun hangi yanıtı verirse versin, kalan olasılıklarda büyük bir azalmayı garanti eden hamledir.

3.2 “Bit” Nedir? ($\log_2$)

Sıkıcı formüllere geçmeden önce, bir **bit**’in gerçekte ne anlama geldiğini ve 2 tabanındaki logaritmanın ($\log_2$) nereden doğduğunu net bir şekilde görselleştirelim.

- **Temel Birim: Tek Bir Evet/Hayır Sorusu**

En temel düzeyde **1 bit**, dengeli tek bir **Evet/Hayır sorusunun** yanıtını aldığımızda kazandığınız bilgi miktarıdır.

1 ile 2 arasında bir sayı tuttuğumu düşünelim:

“Tuttuğum sayı 1 mi?”

İki seçenek de eşit olasılıktaysa, gelen yanıt (“Evet” veya “Hayır”) belirsizliğinizi anında tamamen çözer. Bu tek yanıt size **1 bitlik bilgi** kazandırır. 2 olasılığı 1’e indirir.

- **Evet/Hayır Sorularını Üst Üste Koymak**

Arka arkaya birden fazla Evet/Hayır sorusu sorarsak ne olur?

- **1 Evet/Hayır Sorusu (1 bit):** $2^1 = 2$ olası durumu ayırt edebilir.
- **2 Evet/Hayır Sorusu (2 bit):** İlk soru alanı 2 gruba böler; ikinci soru bu grupları tekrar ikiye böler. Toplamda $2 \times 2 = 2^2 = 4$ olası durumu ayırt eder.
- **3 Evet/Hayır Sorusu (3 bit):** $2 \times 2 \times 2 = 2^3 = 8$ olası durumu ayırt eder.
- **b Evet/Hayır Sorusu (b bit):** 2^b olası durumu ayırt edebilir.

Örüntüyü fark ettiniz mi? Her eklenen 1 bitlik bilgi, eleyebileceğiniz veya ayırt edebileceğiniz olasılık sayısını **ikiye katlar**.

- **Tersine Gitmek: $\log_2$ Nereden Geliyor?**

Şimdi soruyu tersten soralım. b bitin kaç olasılığı yönettiğini sormak yerine, elinizde N adet olasılık olduğunu bildiğinizi varsayalım:

“ N seçenek arasından 1 tanesini bulmak için kaç adet Evet/Hayır sorusuna (bite) ihtiyacım var?”

Çözmemiz gereken denklem şudur: $2^b = N$. Üs olan b 'yi yalnız bırakmak için her iki tarafın 2 tabanında logaritmasını alırlar:

$$b = \log_2(N) \quad (1)$$

2 tabanındaki logaritma işte tam olarak budur! $\log_2(N)$, N **adet seçenek arasından 1 tanesini yakalamak için sormanız gereken ideal Evet/Hayır soru sayısını sayar.**

- **Bunu Wordle’a Uygulamak**

Türkçe Wordle oyunumuzda $N = 2936$ adet muhtemel cevap kelime ile başlarız. Gizli kelimeyi bulmak için toplam ne kadar bilgi toplamamız gerekir?

$$\text{Gerekli Toplam Bilgi} = \log_2(2936) \approx \mathbf{11,52 \text{ bit}} \quad (2)$$

Bu durum, 2.936 aday arasından gizli kelimeyi bulmanın matematiksel olarak yaklaşık **11,52 adet ideal Evet/Hayır sorusunun** yanıtını almaya eşdeğer olduğu anlamına gelir. Wordle’da yaptığımız her tahmin, tek bir turda bu 11,52 bitin olabildiğince fazlasını toplamaya çalışan bir sorudur.

3.3 Olasılık Kovaları

Kalan aday kelimelere ($\mathcal{C}_k \subseteq \mathcal{A}$) karşı bir $g \in \mathcal{V}$ tahminini test ettiğinizde, her aday kelime 243 olasılık renk örüntüsü “kovasından” birine düşer.

Örneğin, 100 aday kelimenin kaldığını varsayalım:

- Kova A (örüntü 02010: 1 yeşil, 1 sarı, 3 gri) 50 kelime alır ($P = \frac{50}{100} = 0,50$).
- Kova B (örüntü 22000: 2 yeşil, 3 gri) 25 kelime alır ($P = \frac{25}{100} = 0,25$).
- Kova C (örüntü 10202: 2 yeşil, 1 sarı, 2 gri) 25 kelime alır ($P = \frac{25}{100} = 0,25$).

Bir p örüntü kovası N_p adet kelime içeriyorsa, o p örüntüsünü almak şu miktarda bilgi sağlar:

$$I(p) = \log_2 \left(\frac{|\mathcal{C}_k|}{N_p} \right) = -\log_2 P(p) \quad \text{bit bilgi.} \quad (3)$$

Daha küçük kovaların daha fazla bit bilgi sağladığına dikkat edin (eğer bir kovada sadece 1 kelime varsa, $I = \log_2(100/1) = 6,64$ bit olur ve anında kazanırsınız!).

3.4 Shannon Entropisi ($H(g)$)

Wordle’a bir tahmin yazdığınızda, oyunun size tam olarak hangi renk örüntüsünü döndüreceğini önceden bilemezsiniz. Bir örüntü geride 50 aday bırakabilirken, başka bir örüntü sadece 2 aday bırakabilir.

Bir tahmini oynamadan önce ne kadar iyi olduğunu değerlendirmek için, o tahminin sağlayacağı genel **beklenen bilgi kazancını** ölçecek bir yöntem ihtiyacımız vardır.

Neden Ağırlıklı Ortalama?

Bir tahminin birden fazla olası sonucu varsa, o tahminin genel değeri, her bir sonuçtan elde edilen bilgi kazancının o sonucun gerçekleşme olasılığıyla ağırlıklandırılmış **ağırlıklı ortalamasıdır**:

$$\text{Beklenen Bilgi} = \sum (\text{Örüntü } p\text{'nin Olasılığı}) \times (\text{Örüntü } p\text{'den Kazanılan Bilgi})$$

Formülde Neden Eksi İşareti (−) Var?

Bölüm 3.3'ten hatırlayacağınız üzere, $P(p)$ olasılığına sahip bir p örüntüsünden kazanılan bilgi şöyle tanımlanır:

$$I(p) = \log_2 \left(\frac{1}{P(p)} \right) = -\log_2 P(p) \quad (4)$$

$P(p)$ değeri 0 ile 1 arasında bir kesir olduğu için (örneğin $P(p) = 0,25$), logaritması $\log_2(0,25) = -2$ her zaman **negatif bir sayıdır**.

Ancak bilgi, belirsizlikteki pozitif bir azalmayı temsil eder (-2 bit değil, $+2$ bit bilgi kazanırız). Negatif logaritmayı pozitif bir bilgi miktarına dönüştürmek için logaritmanın önüne bir **eksi işareti** ($-$) koyarız.

Shannon Entropisi Formülü

Olasılık ağırlıklandırması olan $P(p)$ ile pozitif bilgi kazancı olan $-\log_2 P(p)$ birleştirildiğinde **Shannon Entropisinin** matematiksel tanımı elde edilir:

$$H(g | \mathcal{C}_k) = - \sum_p P(p) \log_2 P(p) \quad (5)$$

Shannon Entropisi, g tahminini oynamanın tüm olası renk örüntüleri genelinde sağlayacağı beklenen bilgiyi (bit cinsinden) temsil eder.

Örnek Bir Hesaplama

Bölüm 3.3'teki 100 kelimelik örneğimizde olasılıklar $P = [0, 50, 0, 25, 0, 25]$ idi:

$$\begin{aligned} H(g) &= - [0, 50 \log_2(0, 50) + 0, 25 \log_2(0, 25) + 0, 25 \log_2(0, 25)] \\ &= - [0, 50(-1) + 0, 25(-2) + 0, 25(-2)] \\ &= 0, 50 + 0, 50 + 0, 50 = \mathbf{1, 50} \text{ bit} \end{aligned}$$

Ortalamada g tahminini oynamak, 100 kalan kelimeyi $\frac{100}{2^{1,50}} \approx 35$ kelimeye düşürecektir. Daha yüksek entropi değeri, daha iyi bir tahmin demektir.

4 Neden Tüm Sözlüğü Aramak Sadece Muhtemel Cevapları Aramaktan İyidir?

Yaygın bir içgüdü, yalnızca gizli kelime olma ihtimali devam eden kelimeleri ($\mathcal{C}_k$) tahmin etmektir. Ancak **tüm geçerli tahmin sözlüğünü** ($\mathcal{V}$) aramak genellikle çok daha güçlü sonuçlar verir. Bunu aşağıdaki örnek üzerinde açıklayalım.

4.1 AKER Tuzak Örneği

4 aday kelimenin kaldığını varsayalım:

$$\mathcal{C}_k = \{\text{BAKER, FAKER, MAKER, TAKER}\}$$

- **Adaylar Arasından Tahmin Yapmak** (örn. $g = \text{BAKER}$):

- %25 doğru olma ihtimali (hepsi yeşil).
- %75 yanlış olma ihtimali (1 gri, 4 yeşil), geride 3 kelime bırakır.
- Beklenen Entropi: **0,81 bit**.

- **Tüm Sözlükten Tahmin Yapmak** (örn. B, F, M, T harflerini içeren bir kelime):
 - Aynı anda 4 harfi birden test eder. Her aday kelime benzersiz bir örüntü üretir (her biri %25 olasılık).
 - Beklenen Entropi: **2,0 bit** ($\log_2(4) = 2, 0$).
 - **Sonuç:** Bir sonraki turda gizli kelimeyi bulmayı %100 kesinlikle garanti eder.

4.2 Aday Eşitlik Bonusu

Dışarıdan bir $g_{\text{ext}} \in \mathcal{V} \setminus \mathcal{C}_k$ kelimesi ile aday bir $g_{\text{cand}} \in \mathcal{C}_k$ kelimesi eşit entropi ürettiğinde, g_{cand} kelimesine anında kazanma şansı bulunduğu için hafif bir öncelik veririz:

$$S(g) = H(g \mid \mathcal{C}_k) + \epsilon \cdot \mathbb{I}(g \in \mathcal{C}_k) \quad (6)$$

burada $\epsilon = 0,0001$ 'dir.

5 Algoritma Sözde Kodları

Algorithm 1 Geri Bildirim Örüntü Eşleştiricisi $f(g, w)$

```

1: Girdi: Tahmin kelimesi  $g$ , Gizli aday  $w$ 
2: Örüntü dizisini ilklendir:  $p \leftarrow [0, 0, 0, 0, 0]$ 
3:                                     ▷ 1. Aşama: Yeşilleri İşaretle (tam eşleşmeler)
4: for  $i \leftarrow 0$  to 4 do
5:   if  $g[i] = w[i]$  then
6:      $p[i] \leftarrow 2$ ;  $g[i]$  ve  $w[i]$  elemanlarını eşleştirdi olarak işaretle
7:   end if
8: end for
9:                                     ▷ 2. Aşama: Sarıların İşaretlenmesi (kısmi eşleşmeler)
10: for  $i \leftarrow 0$  to 4 do
11:   if  $g[i]$  eşleşmemiş ve  $g[i] \in w$  (eşleşmemiş) then
12:      $p[i] \leftarrow 1$ ;  $w$ 'deki eşleşen harfi işaretle
13:   end if
14: end for
15: return  $p_0 p_1 p_2 p_3 p_4$ 

```

Algorithm 2 En Optimal Kelimeyi Bulucu $\text{findBestWord}(\mathcal{C}_k, \mathcal{V})$

```

1: Girdi: Hedef Adaylar  $\mathcal{C}_k \subseteq \mathcal{A}$ , Geçerli Tahmin Sözlüğü  $\mathcal{V}$ 
2: if  $|\mathcal{C}_k| \leq 2$  then
3:   return  $\arg \max_{g \in \mathcal{C}_k} H(g \mid \mathcal{C}_k)$ 
4: end if
5:  $S_{\max} \leftarrow -1$ ,  $g^* \leftarrow \mathcal{C}_k[0]$ ,  $H^* \leftarrow 0$ 
6: for her  $g \in \mathcal{V}$  do
7:   Denklem (3)'ü kullanarak  $H(g \mid \mathcal{C}_k)$  değerini hesapla
8:    $S(g) \leftarrow H(g \mid \mathcal{C}_k) + (\text{eğer } g \in \mathcal{C}_k \text{ ise } 0,0001 \text{ aksi halde } 0)$ 
9:   if  $S(g) > S_{\max}$  then
10:     $S_{\max} \leftarrow S(g)$ ;  $g^* \leftarrow g$ ;  $H^* \leftarrow H(g \mid \mathcal{C}_k)$ 
11:   end if
12: end for
13: return {kelime:  $g^*$ , entropi:  $H^*$ }

```

6 Deneysel Sonuçlar: En İyi Türkçe Başlangıç Kelimeleri

$\mathcal{V}$ kümesindeki 5.500 geçerli tahmin kelimesinin tamamını $\mathcal{A}$ kümesindeki 2.936 hedef kelimeye karşı değerlendirmek en optimal başlangıç kelimelerini verir. Tablo 1’de detaylandırıldığı üzere, **TARİK**, **MERAK** ve **KENAR** tüm veri kümesinde en yüksek bilgi kazancını elde ederek herhangi bir hedef kelimeyi ortalama sadece 3,58 turda çözmektedir.

Table 1: En Optimal Türkçe Başlangıç Kelimeleri

Sıra	Kelime	Shannon Entropisi (Bit)	Ort. Tahmin ($\mathbb{E}[N]$)	Alan Daraltma
1	TARİK	5,8083	3,58 tur	%97,1 (2936 $\rightarrow \approx 52$)
2	MERAK	5,8057	3,58 tur	%97,0 (2936 $\rightarrow \approx 53$)
3	KENAR	5,7868	3,59 tur	%97,0 (2936 $\rightarrow \approx 53$)
4	SERAK	5,7801	—	%96,9
5	KAMER	5,7785	—	%96,9
6	KELAM	5,7379	—	%96,5
7	KALEM	5,7162	—	%96,4
8	TALİK	5,7081	—	%96,4
9	KETAL	5,6973	—	%96,3
10	KEMAL	5,6883	—	%96,3

7 Sonuç

Bilgi Teorisi ve Shannon Entropisi maksimizasyonunu $\mathcal{V}$ tahmin sözlüğünün tamamı üzerinde kaldıraç olarak kullanan **Wordle TR** çözücüsü, binlerce olasılığı ortalama 3,6 turun altında tek bir gizli kelimeye indirmeyi başarmaktadır.